Extending Programmable Blockchain Protocols to Enable and Incentivize Parallel Execution of Smart Contracts

Togzhan Barakbayeva[1], Xuran Cai[2], Zhuo Cai[1], and Amir Goharshady[2]

[1] The Hong Kong University of Science and Technology, Hong Kong, China
tbarakbayeva@connect.ust.hk, zcai@connect.ust.hk
[2] University of Oxford, Oxford, UK
xuran.cai@cs.ox.ac.uk, amir.goharshady@cs.ox.ac.uk

Abstract. Modern nodes on blockchain networks are typically deployed on multi-core hardware. However, in current blockchain consensus protocols, such as those of Cardano and Ethereum, the nodes have to process the transactions in each block in a sequential manner according to the order specified by the producer, leaving their parallel computational resources underutilized. The main challenge in parallel execution comes from transaction dependencies, e.g. different functions reading from and writing to the same location in a contract's storage. This makes safe concurrent execution difficult. A node that wishes to use concurrency currently has to pay the overhead of finding a suitable concurrent schedule. More importantly, since these protocols do not assume parallel execution, the amount of computation performed in each block has to be limited by the sequential computational power of the weakest nodes. Thus, any benefit obtained from parallelization does not translate to shorter block times or higher throughput.

In this work, we solve these problems by presenting two novel parallel blockchain protocols that extend the standard eUTXO and account-based models and provide native support for parallelization. In our protocols, the burden of finding an efficient concurrent schedule is shifted to the producers, who typically have much more computational power than the nodes. Moreover, the producers are incentivized to effectively utilize parallelization and minimize the total runtime (makespan). As a result, the computational limits in each block and the times between any two consecutive blocks can be set according to the *parallel* (rather than sequential) computational power of our nodes, leading to significant improvements in throughput. The full version of the paper is available at HAL hal-05724231[3].

Keywords: Blockchain, Consensus Protocols, Parallel Execution, Concurrent Scheduling

[3] Barakbayeva, T., Cai, X., Cai, Z., Goharshady, A.K.: Extending programmable blockchain protocols to enable and incentivize parallel execution of smart contracts (long version). HAL (2026)

1 Introduction

Sequential Execution. Programmable blockchain protocols propagate new blocks at fixed time intervals, called slots. In Ethereum [6], a slot lasts 12 seconds; in Cardano [16], 1 second. Once a new block is proposed, nodes in the network are responsible for executing transactions in the block against their local copy of the ledger and verifying their validity. All nodes of the blockchain network must finish this process before the next block is proposed, i.e., by the end of the slot. Moreover, when executing the block, nodes should strictly follow the particular order of transactions specified by the producer, in order to ensure consistent state updates. In current architectures, this is done sequentially, on a single core, making transaction processing on nodes highly inefficient. This is a major scalability bottleneck and leads to gross underutilization of the nodes' computational resources.

Previous Works. To address this issue, two lines of solutions have been proposed. One line is to scale at the consensus level by sharding the blockchain into multiple sub-chains [17,21,32] or by adopting DAG/blockDAG-based ledger structures [18,19,30]. The other line is to scale at the execution level by asking each node to utilize modern multi-core processors to execute sequentially ordered transactions in parallel, usually by employing deterministic concurrency controls based on transaction dependency graphs [4,12] or Optimistic Concurrency Control (OCC)-style execution engines [10,20]. Although the related literature has shown great progress on parallel transaction execution, parallelization is not natively supported at the base protocol level. This leads to the following two issues:

Throughput. Firstly, while some nodes might invest in finding an efficient concurrent schedule and execute transactions in parallel, other nodes could still be processing transactions in a sequential order. The execution throughput is therefore limited by the *sequential* computational power of the weakest nodes, i.e., nodes that satisfy only the minimum hardware requirements for participating in the protocol. As a result, the interval between two consecutive blocks (slot time) cannot be reduced and the amount of computation in a fixed slot time cannot be increased. Our approach, on the contrary, provides native support for parallelization that allows shorter block intervals and improved throughput.

Burden of Optimization. Secondly, the burden of computing an optimal or efficient parallel schedule is placed on the nodes themselves, since faster validation is assumed to be in their interest, although no monetary incentives are given. This can be computationally burdensome for the nodes with minimal resources. Moreover, the same optimization task is solved repeatedly by every node, which is wasteful. We alleviate these issues by shifting the workload of constructing a good or optimal scheduling strategy to the producers, who also have much more processing power. In our design, producers are monetarily incentivized to propagate the schedule for multi-core execution along with the block, making it possible for the nodes to follow the parallel strategy and achieve better execution performance.

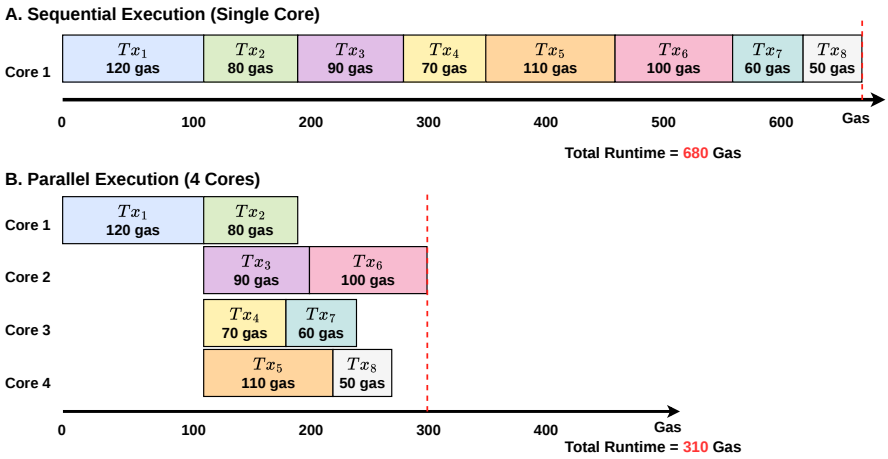


Fig. 1: An example of a parallel transaction execution strategy.

Example. Figure 1 illustrates a sample parallel schedule across multiple cores that is proposed by the producer, significantly reducing the total runtime, i.e., makespan, compared to the sequential version. Our mechanism is embedded at the protocol level, making it feasible for every node in the network to access the parallel schedule and follow it. This not only removes the overhead of finding parallelization strategies for nodes, but also leads to a shorter slot duration.

Makespan Minimization. When transactions are executed in parallel across multiple cores, the objective is to minimize the makespan: the finishing time of the last core to complete its assigned workload. This is known as the makespan minimization problem in scheduling theory [11]. However, the blockchain context introduces additional constraints that make the traditional problem setting much more complex. First, transactions might have dependencies. In programmable blockchains based on the eUTXO [7] and account models [6], this is even more pronounced. In the eUTXO model, e.g., on Cardano, one transaction A can use the output of another transaction B as an input, leading to the condition that B must be executed before A, be it across multiple cores or within a single core. In the account model, for some transactions, concurrent execution may be infeasible. For example, two Ethereum transactions A and B might access/modify the same storage slots. Therefore, parallel execution of these transactions could result in undesirable behavior. An example of the dependency graph is given in [5].

Gas. In programmable blockchains that support smart contracts, a transaction consumes computational resources. In order to track this and prevent denial-of-service attacks, the concept of *gas* was introduced by Ethereum [6]. Each atomic operation is assigned a predefined gas consumption, which is a rough measure of the real-world cost of executing it, and the users have to pay for the total

gas consumed by their transactions/function calls. In practice, the user gets to decide the amount of currency they are willing to pay *per unit of gas* and the producers/validators can accordingly decide whether to include the transaction in their block.

Gas as Time. In this work, when assigning transactions to the cores, our protocol uses gas as a measure of time to define the start and end points of a transaction, i.e., in the sequel, we use the terms "gas" and "time" synonymously. This is because the contents of a block, messages sent in the protocol, and rewards must be machine-independent and subject to consensus. Gas already provides such a machine-independent measure for the cost of executing an operation or program.

Gas Limits and Minimum Hardware Requirements. In our multi-core setting, we set a total gas limit G for all transactions that are assigned to the same core. To make the published schedule meaningful, the protocol should also announce a minimum hardware requirement for validators: a validator is expected to have at least K physical cores, or, more generally, enough aggregate computational capacity to execute the published K-core schedule within the slot duration. In other words, the requirement is not merely that a validator have some abstract "capacity," but that it be able to follow the producer's declared per-core plan in real time. A validator with fewer than K physical cores may still participate, but it cannot be expected to keep up with the published schedule and may fall behind the latest block. This fits nicely into our usage of gas as a machine-independent approximation of running time.

Our Contribution. In this work, we propose two novel parallel protocols for programmable blockchains, one extending the eUTXO model and the other extending the account model. In both protocols, the producers/validators are put in charge of the task of synthesizing and publishing efficient parallel schedules and are rewarded for this task. More specifically, they are monetarily incentivized to produce blocks with a small makespan. Our protocols allow a blockchain network node with minimal hardware resources to efficiently verify the validity of the producer's proposed schedule. They ensure that (i) the slot time is dictated by the *parallel* computational power of the weakest nodes, rather than their *sequential* computational power, and (ii) the weakest nodes do not fall behind and can keep up with the consensus chain. Our protocols simply change the contents and fields present in each block. Thus, they can be run on top of any consensus mechanism, such as proof-of-work or proof-of-stake. We do not modify the consensus layer.

2 Related Work

Parallel execution engines for blockchains. A large body of recent work studies how validators can exploit multi-core hardware while preserving the deterministic semantics required by replicated blockchains. Block-STM [10] is a prominent optimistic execution engine: it speculatively executes

transactions in parallel, validates them against a preset block order, and re-executes transactions whose reads become invalid. ParallelEVM [20] refines this line of work by resolving conflicts at the operation level rather than aborting whole transactions, using an SSA-style operation log to identify only the operations affected by a conflict. Garamvölgyi et al. [9] complement these execution-engine designs with an empirical study of Ethereum workloads, showing that application-inherent conflicts, such as hot counters and highly contended state, can sharply limit naive transaction-level parallelism. Shahid's thesis [28] provides a broader systems treatment of parallel transaction execution in public blockchains. These works have several limitations: (1) they suffer from the overhead of re-executing conflicted transactions, and (2) their makespan is unknown and uncertain.

Scheduling with auxiliary data. If validators have access to auxiliary data, they might avoid the above limitations. Choices of auxiliary data include read/write access sets of transactions, dependency and/or conflict graphs of transactions, and parallel fork-join schedules. These auxiliary data can come from users and producers. Anjana et al. [4] study block transactional memory algorithms that use conflict specifications, showing that declared conflict metadata can improve parallel execution on EVM and MoveVM workloads. Heimbach et al. [13] analyze Ethereum's EIP-2930 optional access lists and find that, although access lists could in principle help identify independent transactions, their current adoption and accuracy are insufficient for reliable parallelization. The closest work to us is [8], which produces a fork-join schedule to assist validator replay and allows validators to check correctness. Our work further connects such scheduling with producer incentives and block limit constraints.

Makespan optimization and block construction. The closest systems comparator to our scheduling problem is the work of Karmegam, Kiffer, and Fernández Anta [15], which formulates two producer-side optimization problems: minimizing the makespan of an already ordered block on multiple cores, and selecting/scheduling mempool transactions under a runtime limit to maximize validator reward. Their MILP formulations and deterministic heuristics quantify the gap between optimal schedules and scalable scheduling algorithms. Our producer-side problem differs by presenting a per-core gas limit for the producer.

Parallel-execution fees. Parallel execution also changes how transaction costs should be priced. Acilan et al. [1] introduce gas computation mechanisms for parallel execution, including mechanisms that account for storage-key contention and makespan contribution. Ndiaye [23] studies pricing in systems with multiple transaction queues or submarkets under a global capacity constraint, providing an economic model for local congestion and parallel execution fee markets. Wadhwa et al. [31] show that fee mechanisms that depend on parallel execution face additional strategic risks under execution uncertainty: users or schedulers may manipulate contention, and contingent transactions create a trade-off between user risk and scheduler risk. These works are complementary to our protocol. We benefit from a multi-dimensional fee scheme where there

is one dedicated measurement of execution time in consensus. Our work implicitly makes low-contention transactions whose inclusion does not increase the makespan more likely to be included during congestion, without introducing new strategic attacks by both users and schedulers.

Producer-assisted schedules and dependency metadata. Several closely related studies also allow a proposer to publish execution-related metadata to accelerate validator execution. OptSmart [3] studies optimistic concurrent execution for smart contracts and uses compact metadata to reduce the amount of state that validators must track during speculative execution. The study on DAG-based scheduling for blockchains [24] considers producer-side scheduling for Hyperledger Sawtooth-like settings and uses a dependency DAG to improve parallel execution and load balancing across workers. These works also examine the use of dependency information in the block and the effect of incomplete or excessive dependency disclosure by a malicious proposer. They also provide alternative ways to improve load balancing across cores beyond the fork-join schedule of Dickerson et al. [8]. Our protocol differs in that the schedule is not merely advisory metadata but a protocol-level certificate: validators check the certified per-core plan directly rather than reconstructing or repairing a speculative execution trace, and the plan is coupled to block validity and producer incentives.

Read-write-aware and read-write-oblivious execution. A related theoretical line of work studies read-write-aware and read-write-oblivious execution and characterizes the complexity of parallel smart-contract execution. The work of block transactional memory [2] introduces complexity-theoretic and algorithmic perspectives on this problem, including preset-based serializability and single-version and multi-version execution algorithms. Our protocol differs by making the producer's schedule explicit and verifiable up front, which yields stronger validation guarantees and a cleaner bound on the amount of work that validators must perform. In particular, unlike STM-style approaches, our protocol avoids re-execution on the validator side and therefore offers a substantially better theoretical bound for block validation.

Implementation-oriented systems. The implementation-oriented literature on parallel execution also includes systems such as Solana's Sealevel [29], Hyperledger Sawtooth [14], Monad's parallel execution engine [22], the parallel-EVM efforts of Sei [27] and Polygon [25], and RiseChain's PEVM [26]. These systems are primarily execution-engine or runtime designs and are often optimized for practical throughput and compatibility with existing virtual machines. Our protocol is not meant to replace such engines; rather, it provides a protocol-level abstraction that couples scheduling, validation, and incentives. In this sense, it complements execution engines by making producer-assisted parallelism part of the block validity interface and by reducing the validator-side burden of speculative replay.

3 General Protocol Framework

In our prototype framework, we assume a fixed K-core execution environment. Instead of publishing a single sequential order, the producer generates a parallel replay schedule together with additional scheduling metadata, and validators directly replay transactions according to the scheduling information stored in the block.

General setting. Let $\mathcal{T} = \{\mathsf{tx}_1, \ldots, \mathsf{tx}_n\}$ denote the set of pending transactions available to the producer. For each transaction tx_i, we write g_i for its gas cost and f_i for its transaction fee.

Block Structure. Besides the original blockchain data, each block additionally stores scheduling-related metadata for deterministic parallel execution.

Formally, let a block be represented as:

$$B = (\mathsf{Header}, \mathsf{ConsensusData}, \mathsf{Receipt}, \mathsf{StateRoot}, \mathsf{Schedule}, \pi),$$

where:

- Header contains standard block header information such as parent hash, timestamp, block number, and producer information;
- ConsensusData contains consensus-related metadata;
- Receipt stores transaction execution receipts;
- StateRoot denotes the final state commitment after block execution;
- Schedule denotes the parallel scheduling metadata;
- π denotes a serial witness order with begin/end actions of transactions in the block.

Scheduling Metadata. The scheduling metadata is defined as

$$\mathsf{Schedule} = (\mathsf{Schedule}_1, \mathsf{Schedule}_2, \ldots, \mathsf{Schedule}_K),$$

where each $\mathsf{Schedule}_k$ corresponds to the transaction series assigned to execution core k, given by an ordered sequence of N_k scheduled transactions

$$\mathsf{Schedule}_k = \big((\mathsf{tx}_1^k, b_1^k, e_1^k), \ldots, (\mathsf{tx}_{N_k}^k, b_{N_k}^k, e_{N_k}^k)\big).$$

Here, each record consists of:

- tx_i^k, the transaction content;
- b_i^k, the begin execution time on core k;
- e_i^k, the end execution time on core k.

Gas-time synchronization. For every scheduled transaction tx_i^k with begin time b_i^k and end time e_i^k on core k, define its begin and end actions

$$B_i = (b_i^k, \mathsf{begin}, i), \qquad E_i = (e_i^k, \mathsf{end}, i),$$

and let

$$\mathcal{A} = \{B_i, E_i : \mathsf{tx}_i^k \text{ is scheduled}\}$$

be the set of all actions. Actions are ordered by gas time: of two actions with different gas times, the one with the smaller gas time is delivered first; if they share a gas time and one is an end and the other a begin, the end is delivered first; actions of the same type at the same gas time may be delivered in any order. Merging the action lists of all K schedules in this order yields the total order

$$\pi = (a_1, a_2, \ldots, a_{2N})$$

over $\mathcal{A}$, the serial witness order, where $N = \sum_k N_k$ is the number of scheduled transactions. We write $a <_\pi a'$ if action a occurs before action a' in π; since $b_i^k < e_i^k$ always, $B_i <_\pi E_i$ for every scheduled transaction.

Cross-core dependencies and ordering scope. Transactions assigned to the same core are executed sequentially by construction, while dependencies across different cores are enforced by the published begin/end times together with the dependency checks in validator replay. In particular, for every dependent or conflicting pair (i, j) (edge in D), the schedule must satisfy $e_i \leq b_j$ when i must precede j; if this is violated, validators reject the block. No such ordering is required for non-conflicting pairs, which may overlap and execute in parallel. Thread safety is enforced by shared synchronization state during replay (e.g., the shared UTXO active-use table in the UTXO protocol and shared access-status map/barriers in the account protocol), with begin/end checks and atomic commit points preventing inconsistent shared-state updates.

Block Limitation. Our design introduces a per-core block limitation mechanism. Instead of enforcing a single global gas or transaction-size limit for the entire block, as adopted in traditional blockchain protocols, we allocate an independent execution budget to each core. Specifically, each core is constrained by a per-core gas budget G that upper-bounds the runtime resources consumed during parallel execution on that core.

Producer incentive. The producer collects the transaction fees of the included transactions as its reward. Under the fixed K-core model the block budget is a per-core gas limit, so to earn more fees the producer is driven to pack as many transactions as possible across the K cores — which is exactly makespan minimization, since poor scheduling strands capacity on a single core and leaves room for only $1/K$ of the block. This incentive already exists even without any extra reward, because a block that uses the available per-core budget more effectively can include more transactions and therefore collect more fees.

To preserve the incentive when the mempool does not contain enough transactions to fill the block, the protocol can use a two-stage reward rule. In the first stage, the producer is rewarded only with the ordinary transaction fees and the block reward; this already encourages it to exploit the per-core budget efficiently. In the second stage, if the block remains underfilled after the producer has maximized the number of includable transactions, the protocol may add a small supplementary reward for the saved execution time,

$$\mathsf{reward}_{\mathrm{parallel}} = \Delta t \times \gamma,$$

where Δt is the execution time saved relative to sequential execution and γ is a protocol-defined parameter. This design keeps the main reward channel tied to transaction inclusion and block filling, while using the extra term only as a secondary incentive for schedule quality when the base incentive is otherwise weak. The parameter γ can be chosen so that the supplementary reward is smaller than the fee of any transaction, ensuring that omitting a transaction remains unprofitable.

Malicious or faulty schedules. The protocol assigns schedule construction to the producer but keeps validity checks at the validators. Missing dependencies, incorrect timing claims, inconsistent receipts, and false access metadata are detected during validator replay and cause block rejection, so safety is preserved. From an incentive perspective, a rational producer has no profitable reason to publish such a block: once rejected, it loses both fees and block reward for that slot. A schedule that is intentionally sequential is still valid, but it is economically dominated under normal demand because it underuses the per-core budget and leaves transaction fees uncollected relative to a better parallel schedule. In low-demand regimes, where fee pressure alone may be weak, the supplementary makespan reward above restores the incentive to avoid low-quality schedules. The practical cost of an invalid schedule is therefore performance rather than safety: validators may spend part of a slot before rejection. This is analogous to existing denial-of-service pressure from invalid blocks, and can be mitigated in practice by ordering checks from cheap to expensive (header/syntax/schedule consistency before replay) and by consensus-layer penalties for repeatedly proposing invalid blocks.

Experiments. We evaluate the protocol by replaying real blocks and scheduling all of their transactions across 8 cores without modifying transactions in the blocks; the schedules already yield a 2.2× gas speedup on Cardano and a 3.84× speedup on 8-core Ethereum, and we expect larger gains under a per-core gas limit, which would incentivize producers to pack more parallelism into each block. Full setup, data, and results are in the full version of the paper [5].

4 Protocols for UTXO-Based Systems

General setting. In the UTXO model, every transaction has a fixed set of input and output UTXOs, given as part of the transaction data. Let $\mathcal{U}_0$ denote the set of unspent transaction outputs in the ledger state before the block is executed. For each transaction tx_i, the transaction data already specifies its input set I_i, its output set O_i, its gas cost g_i, and its transaction fee f_i. In this model, we assume that g_i is fixed and known when the producer constructs the block.

Dependency data. The dependency relation is derived directly from the UTXO references. If transaction tx_i creates an output that is later consumed by transaction tx_j, then tx_i must be executed before tx_j. Formally, we define the dependency edge set

$$D = \{(i, j) : O_i \cap I_j \neq \emptyset\}.$$

The producer must also respect the usual double-spending rule. If two distinct transactions consume the same UTXO, then they cannot both appear in the same valid block. We capture this by the conflict set

$$C = \{\{i, j\} : i < j \text{ and } I_i \cap I_j \neq \emptyset\}.$$

If an input of tx_i is neither in $\mathcal{U}_0$ nor produced by another transaction in the pending transaction set $\mathcal{T}$, then tx_i is unavailable for the current block and must not be selected. Otherwise, if an input is produced by another available transaction, the corresponding dependency edge in D forces that producer to be selected whenever tx_i is selected.

Producer Optimization. Given a set of candidate transactions, the producer decides which transactions are included in the block, which execution core each transaction is assigned to, the execution order of transactions on each core, and the start and end time of each transaction, while satisfying all dependency constraints. Constraints require that (i) every transaction executes after all of its predecessors, (ii) transactions assigned to the same core execute sequentially without overlap, and (iii) each execution core is subject to the per-core gas budget. The objective is to maximize block revenue while minimizing the overall execution time (makespan). Formally, this optimization problem can be expressed as an Integer Linear Program (ILP). The complete formulation is provided in [5].

4.1 Validator Replay

When a validator receives a block B, it performs the ordinary header and consensus-data checks, and then validates that the scheduled execution produces the claimed receipts and final state root. The main extra issue is that gas time and wall-clock running time need not coincide. We therefore treat every scheduled begin time and end time as a logical action, and synchronize these actions by gas time.

Replay algorithm. The synchronized validator maintains a shared UTXO state UTXO and a shared active-use table Active. The key execution rule is that transactions never access shared state while they are running. At a begin action, the current core accesses the shared state to validate and reserve the transaction's UTXOs and pulls the state associated with its inputs into a private execution buffer. The transaction then executes entirely against this private state: all reads are served from the local buffer, and all state changes are kept locally until the transaction reaches its end action. At the end action, after execution and ordinary validation succeed, the current core atomically pushes the transaction's committed changes from its private buffer to the shared UTXO state. Thus, all accesses to shared memory occur only at begin and end actions; no shared-memory access occurs during the interior of a transaction execution.

The active-use table records which UTXO identifiers are currently reserved by transactions that have begun but not yet ended. A more explicit validator-side procedure is in Figure 2. A running example is included in the full version [5].

Procedure ValidateUTXOReplay($B, \pi, \mathcal{U}_0, \mathsf{Schedule}, \mathsf{Receipt}, \mathsf{StateRoot}$)
Input: block B, serial witness order π, pre-block UTXO set $\mathcal{U}_0$, per-core schedule Schedule, claimed receipts Receipt, final state root StateRoot.
Output: accept or reject; if accepted, return the final UTXO state and receipts.
Shared-memory variables: UTXO and Active. They are accessed only at begin and end actions. At begin, the executing core pulls the required UTXO state into a private buffer; during execution, all state access and changes are local; at end, the core pushes the buffered changes to shared memory.

1. **Check block validity.** Validate the block header, consensus data, transaction syntax, and the claimed pre-block state root. Parse $\mathsf{Schedule} = (\mathsf{Schedule}_1, \ldots, \mathsf{Schedule}_K)$ and verify that each selected transaction appears exactly once and that each record satisfies $0 \le b_i^k < e_i^k \le G$ and $e_i^k - b_i^k = g_i$. Check that the serial witness order π follows the schedule.
2. **Initialize.** Set $\mathsf{UTXO} \leftarrow \mathcal{U}_0$ and $\mathsf{Active} \leftarrow \emptyset$. For every transaction tx_i, allocate a private buffer Buf_i for its input state, local changes, and receipt.
3. **Execute in parallel.** Process actions following π. For each action $a \in \pi$:
   (a) **Precondition.** a is either $(b_i^k, \mathsf{begin}, i)$ or (e_i^k, end, i).
   (b) **If** $a = (b_i^k, \mathsf{begin}, i)$ **then**
      i. wait until every earlier action in π has finished;
      ii. if some $u \in I_i$ is not in UTXO, reject;
      iii. if some $u \in I_i \cup O_i$ is already in Active, reject;
      iv. otherwise mark every $u \in I_i \cup O_i$ active, reserve inputs I_i, and pull the state of I_i from UTXO into Buf_i;
      v. start execution of tx_i using only Buf_i. Until its end action, all state access and changes remain local to Buf_i.
   (c) **Else if** $a = (e_i^k, \mathsf{end}, i)$ **then**
      i. wait until execution of tx_i has completed;
      ii. if ordinary validation fails, $g_i \neq e_i^k - b_i^k$, or the receipt differs from the claimed receipt, reject;
      iii. if some $u \in I_i \cup O_i$ is still active for another transaction, reject;
      iv. otherwise push the changes in Buf_i to shared state: mark inputs in I_i as spent, reject if some output in O_i already exists in UTXO, add outputs in O_i to UTXO, remove $I_i \cup O_i$ from Active, and store the receipt.
4. **Finalize.** Accept iff all actions are processed and the stored receipts equal Receipt and the final UTXO commitment equals StateRoot.

Fig. 2: Procedure: Validator-side replay for the UTXO-based protocol.

Complexity. Let $N = \sum_k N_k$ be the number of scheduled transactions, let $S_{\mathrm{utxo}} = \sum_i (|I_i| + |O_i|)$ be the total UTXO metadata size, let $B = \sum_i g_i$ be the total execution gas. The schedule scan costs $O(N)$ work. The UTXO-table and active-use checks cost $O(S_{\mathrm{utxo}})$ expected work with hash tables, or the same asymptotic work after deterministic partitioning by UTXO identifier. Transaction execution costs $O(B)$ gas work. Thus the total validation work is $O(B + S_{\mathrm{utxo}} + N)$, plus the ordinary cost of checking the final state commitment.

With K validator cores, balanced metadata processing, and parallel state-commitment support, the parallel validation time is $O\left(\frac{B+S_{\text{utxo}}}{K}+N\right)$, again plus the final commitment check.

Correctness. We present Lemma 1 and Theorem 1 informally. Formal treatment and detailed proofs can be found in the full version [5].

Lemma 1 (Action-prefix invariant). *At every point during replay, as long as the block has not yet been rejected, the validator's state matches an honest sequential run: the shared UTXO ledger reflects exactly the transactions that have finished so far, applied in their finishing order; the active-use table reflects exactly the UTXOs held by transactions currently mid-execution; and each such transaction's private buffer holds exactly its own inputs plus its own changes so far.*

Theorem 1 (Soundness of validator replay). *If the validator accepts a block B, then the validator replay is equivalent to the valid sequential execution of the scheduled transactions in the order each transaction finished during replay. In particular, this sequential execution produces exactly the claimed receipts and the claimed final UTXO state.*

5 Protocols for Account-Based Systems

In this section, we consider an Ethereum-style account-based model with Turing-complete smart contracts.

Differences from the UTXO model. While the UTXO model allows inferring transaction dependencies based on the input and output data, in the account model, transaction dependencies can be analyzed only after the execution. The transaction trace and the gas consumed depend on the global state at the time of the execution. Therefore, when a producer views pending transactions in the mempool, it cannot infer the dependency information without executing transactions in a particular order. Once the producer outputs a valid schedule, its execution semantics is fixed: it coincides with the sequential execution of the transactions in some particular order. The producer and validators therefore compute the same dependencies for each transaction, and a validator can verify the schedule by executing the transactions following the schedule and checking, against the dependencies it computes during execution, that the received schedule introduces no conflict.

State information. In an account-based model, we refer to the global state as a state key, which consists of an account balance, an account nonce, and a contract-storage location. A transaction might read a state key or write it during execution, and read/write sets cannot be determined in advance by the transaction syntax alone. A smart-contract transaction may branch on the current global state, and hence its gas usage and state accesses are only revealed by execution.

Dependency data. The producer fixes a serial transaction order σ for the selected transactions and executes them sequentially in this order from the pre-block state. The transaction dependency relation in the account model is the conflict relation induced by the read and write sets. If tx_i precedes tx_j in σ, then tx_j depends on tx_i whenever the two transactions access a common state key and at least one of them writes that key. Formally,

$$D^{\sigma}_{\text{account}} = \{(i,j) : i <_{\sigma} j \text{ and } (W_i \cap R_j) \cup (W_i \cap W_j) \cup (R_i \cap W_j) \neq \emptyset\}.$$

Any schedule that overlaps two transactions connected by such an edge may observe a state that differs from the serial execution in order σ. Thus, the schedule must place every edge $(i,j) \in D^{\sigma}_{\text{account}}$ in temporal order, i.e., $b_j \geq e_i$.

Producer Optimization. As in the UTXO case, the producer jointly determines transaction inclusion, core assignment, and execution scheduling in order to maximize block revenue while minimizing the resulting makespan. The ILP formulation (differing only in how dependency information is computed) is provided in [5].

5.1 Validator Replay

The validator receives a block B containing the selected transactions, the serial witness order π, the per-core schedule $\mathsf{Schedule}$, the claimed receipts, and the claimed final state root.

We mainly present the steps to validate and execute $\mathsf{Schedule}$. Parse $\mathsf{Schedule}$ as $(\mathsf{Schedule}_1, \ldots, \mathsf{Schedule}_K)$. A more explicit procedure is given in Figure 3.

This verification does not require the validator to solve the producer's optimization problem. The validator only checks that the published schedule is internally consistent, that the published dependency certificate matches the actual execution, and that the resulting parallel execution produces the claimed block output.

Dependency validation. To verify the schedule during parallel execution, the validators maintain a shared map $A\colon \text{Address} \times \text{Time} \to \text{Status}$, accessible to all cores, where $\text{Status} \in \{\texttt{R}, \texttt{W}, \texttt{FREE}\}$. For each transaction tx_i, its access interval is the half-open interval $[\,\text{begin}(\mathsf{tx}_i),\ \text{end}(\mathsf{tx}_i)\,)$.

In the end action of a transaction tx_i, a validator extracts its read set R_i and write set W_i and, for each accessed address, inspects the slice of A over the interval $[\,\text{begin}(\mathsf{tx}_i),\ \text{end}(\mathsf{tx}_i)\,)$. For each address $a \in R_i$, if

$$A[a]\,[\,\text{begin}(\mathsf{tx}_i),\ \text{end}(\mathsf{tx}_i)\,)$$

already contains the status $\texttt{W}$, then a write–read conflict overlaps tx_i's execution interval and the schedule is rejected; otherwise the validator marks this interval as read,

$$A[a]\,[\,\text{begin}(\mathsf{tx}_i),\ \text{end}(\mathsf{tx}_i)\,) \leftarrow \texttt{R}.$$

Symmetrically, for each address $a \in W_i$, if the same interval contains either $\texttt{R}$ or $\texttt{W}$, the schedule is rejected; otherwise the validator marks the interval as

Procedure ValidateAccountReplay($B, \pi, \mathsf{Schedule}, \mathsf{Receipt}, \mathsf{StateRoot}$)
Input: block B, serial witness order π, per-core schedule Schedule, claimed receipts Receipt, final state root StateRoot.
Output: accept or reject; if accepted, return the final state and receipts.
Shared-memory variables: State and A. They are accessed only at begin and end actions; during transaction execution, all state access and changes remain local to a private buffer.

1. **Basic checks.** Validate the block header, consensus data, transaction syntax, and pre-block state root. Parse $\mathsf{Schedule} = (\mathsf{Schedule}_1, \ldots, \mathsf{Schedule}_K)$. Check that every scheduled transaction has exactly one begin action and one end action in π, consistent with its record $(\mathsf{tx}_i^k, b_i^k, e_i^k)$ in Schedule. For every record, require
$$0 \le b_i^k < e_i^k \le G, \qquad e_i^k - b_i^k = g_i.$$
Check that the serial witness order π follows the schedule.
2. **Initialize.** Set State to the pre-block state and initialize the shared access map A to `FREE`. For every transaction tx_i, allocate a private execution buffer Buf_i.
3. **Execute in parallel.** Process actions in the order π. For each action $a \in \pi$:
   (a) **If** $a = (b_i^k, \mathsf{begin}, i)$**:**
      i. wait until every earlier action in π has finished;
      ii. pull the current state needed by tx_i from State into Buf_i;
      iii. start execution of tx_i using only Buf_i; all reads and writes during execution are local.
   (b) **If** $a = (e_i^k, \mathsf{end}, i)$**:**
      i. wait until execution of tx_i finishes, and derive its read set R_i and write set W_i;
      ii. reject if ordinary validation fails, the observed gas differs from $e_i^k - b_i^k$, or the receipt differs from the claimed receipt;
      iii. for each $x \in R_i$, if $A[x][b_i^k, e_i^k)$ contains `W`, reject; otherwise set
$$A[x][b_i^k, e_i^k) \leftarrow \texttt{R};$$
      iv. for each $x \in W_i$, if $A[x][b_i^k, e_i^k)$ contains `R` or `W`, reject; otherwise set
$$A[x][b_i^k, e_i^k) \leftarrow \texttt{W};$$
      v. atomically push the buffered changes in Buf_i to State and store the receipt.
4. **Finalize.** Accept iff all actions in π are processed, the stored receipts equal Receipt, and the resulting state root equals StateRoot.

Fig. 3: Procedure: Validator-side replay for the account-based protocol.

written,
$$A[a]\,[\,\mathrm{begin}(\mathsf{tx}_i),\ \mathrm{end}(\mathsf{tx}_i)\,) \leftarrow \texttt{W}.$$
A read–read overlap is permitted, as concurrent reads commute and do not affect the resulting state.

The interval is half-open so that an end action and a begin action at the same time do not overlap. In particular, if

$$\text{begin}(\mathsf{tx}_2) = \text{end}(\mathsf{tx}_1) = t,$$

then a read of a by tx_2 at t does not conflict with a write of a by tx_1 committed at its end action at t. The state written by tx_1 is already committed before tx_2 pulls its state at t, so this schedule is valid.

Complexity. Let $N = \sum_k N_k$ be the number of scheduled transactions and $B = \sum_i g_i$ the total execution gas. Scanning the schedule for well-formedness, checking transaction uniqueness, and validating the transaction receipts take $O(N)$ work. Instrumented execution of the scheduled transactions contributes $O(B)$ work. For each transaction, validating its dependencies costs $O(|R_i|+|W_i|)$, where R_i and W_i are its read and write sets; let $S_{\text{account}} = \sum_i \big(|R_i| + |W_i|\big)$ be the total access-setsize over all transactions. Following analysis similar to the UTXO model, the total validation work on K cores is $O\big(\frac{B+S_{\text{account}}}{K} + N\big)$.

Correctness. The correctness argument follows the same structure as the UTXO-based protocol. All standard transaction-validation checks remain unchanged, and any invalid action causes the entire block to be rejected. Therefore, the only additional concern is whether the dependency check correctly captures the potential conflict even with a possible fractured-read problem. In the account model the validator learns each transaction's read and write sets only by executing it. Reads observe live shared state with no snapshot isolation, so two overlapping transactions may be observed differently depending on the timing of a read relative to a commit (a *fractured read*), and two validators can record different access sets for the same block. We present Theorem 2 and provide full proofs in [5].

Theorem 2. *If at least one validator accepts a block, then every validator accepts it, and some serial execution of the scheduled transactions produces exactly the claimed receipts and final state root.*

6 Conclusion and Future Works

We present a general protocol framework for high-throughput programmable blockchains that incentivizes block producers to optimize parallel execution under a per-core gas constraint. Existing work on blockchain parallel execution either treats parallelism as a validator-side optimization, ignoring the trustless and economic structure of blockchain protocols, or does not provide an incentive-compatible mechanism for optimizing validator replay. Our framework moves schedule construction to producers, makes the schedule validator-checkable, and aligns producer rewards with lower parallel makespan.

The practical performance of this framework depends on a consensus-level metric that accurately approximates transaction execution time. Recent work on multidimensional fee schemes is a promising step toward reducing the discrepancy between gas and physical execution time. We leave the design and modeling of such a metric for future work.

Acknowledgments.



This work was partially funded by the European Union through the ERC Grant SPES (101222524) and by the Italian Ministry of Universities and Research (MUR) through the PNRR Mission 4 Grant ERC2024_00004. Togzhan Barakbayeva and Zhuo Cai were supported by the Hong Kong PhD Fellowship Scheme.

Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them. Authors are ordered alphabetically.

References

1. Acilan, B., Constantinescu, A., Heimbach, L., Wattenhofer, R.: Transaction fee market design for parallel execution. In: AFT (2025)
2. Anjana, P.S., Ravi, S.: Block Transactional Memory: A complexity study. In: SSS. pp. 56–75 (2026)
3. Anjana, P.S., et al.: OptSmart: A space-efficient optimistic concurrent execution of smart contracts. Distributed and Parallel Databases **42**, 245–297 (2024)
4. Anjana, P.S., et al.: Efficient parallel execution of blockchain transactions leveraging conflict specifications. In: AFT (2025)
5. Barakbayeva, T., Cai, X., Cai, Z., Goharshady, A.K.: Extending programmable blockchain protocols to enable and incentivize parallel execution of smart contracts (long version). HAL, hal-05724231 (2026)
6. Buterin, V., et al.: A next-generation smart contract and decentralized application platform (2014)
7. Chakravarty, M.M.T., et al.: The extended UTXO model. In: Financial Cryptography Workshops (WTSC) (2020)
8. Dickerson, T.D., Gazzillo, P., Herlihy, M., Koskinen, E.: Adding concurrency to smart contracts. In: PODC. pp. 303–312 (2017)
9. Garamvölgyi, P., et al.: Utilizing parallelism in smart contracts on decentralized blockchains by taming application-inherent conflicts. In: ICSE (2022)
10. Gelashvili, R., et al.: Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. In: PPoPP (2023)
11. Graham, R.L.: Bounds for certain multiprocessing anomalies. Bell System Technical Journal **45**, 1563–1581 (1966)
12. Hay, Y., Friedman, R.: Batch-Schedule-Execute: On optimizing concurrent deterministic scheduling for blockchains. In: SRDS (2024)
13. Heimbach, L., et al.: Dissecting the EIP-2930 optional access lists. In: FC (2024)
14. Hyperledger Sawtooth: Transaction scheduling. https://sawtooth.hyperledger.org/docs/core/nightly/master/architecture/scheduling.html
15. Karmegam, A., Kiffer, L., Anta, A.F.: Exploiting multi-core parallelism in blockchain validation and construction. arXiv (2026)

16. Kiayias, A., Russell, A., David, B., Oliynykov, R.: Ouroboros: A provably secure proof-of-stake blockchain protocol. In: CRYPTO (2017)
17. Kokoris-Kogias, E., et al.: OmniLedger: A secure, scale-out, decentralized ledger via sharding. In: S&P. pp. 583–598 (2018)
18. Lewenberg, Y., Sompolinsky, Y., Zohar, A.: Inclusive block chain protocols. In: FC. pp. 528–547 (2015)
19. Li, C., et al.: Scaling Nakamoto consensus to thousands of transactions per second. CoRR **abs/1805.03870** (2018)
20. Lin, H., Feng, H., Zhou, Y., Wu, L.: ParallelEVM: Operation-level concurrent transaction execution for EVM-compatible blockchains. In: EuroSys (2025)
21. Luu, L., et al.: A secure sharding protocol for open blockchains. In: CCS. pp. 17–30 (2016)
22. Monad: Parallel execution. https://docs.monad.xyz/monad-arch/execution/parallel-execution
23. Ndiaye, A.: Parallel execution fee mechanisms. IACR ePrint (2024)
24. Piduguralla, M., et al.: DAG-based efficient parallel scheduler for blockchains: Hyperledger Sawtooth as a case study. In: Euro-Par. pp. 184–198 (2023)
25. Polygon Labs: Innovating the main chain: A Polygon PoS study in parallelization. https://polygon.technology/blog/innovating-the-main-chain-a-polygon-pos-study-in-parallelization
26. RISE Labs: RISE PEVM: Parallel EVM. https://github.com/risechain/pevm
27. Sei: Sei parallelization engine: Multi-core blockchain execution. https://docs.sei.io/learn/parallelization-engine
28. Shahid, R.: Parallel transaction execution in public blockchain systems. Master's thesis, UWaterloo (2024)
29. Solana Labs: Sealevel — parallel processing thousands of smart contracts. https://solana.com/news/sealevel---parallel-processing-thousands-of-smart-contracts
30. Sompolinsky, Y., Lewenberg, Y., Zohar, A.: SPECTRE: A fast and scalable cryptocurrency protocol. IACR Cryptol. ePrint Arch. **2016**, 1159 (2016)
31. Wadhwa, S., Yaish, A., Zhang, F., Nayak, K.: Perils of parallelism: Transaction fee mechanisms under execution uncertainty. In: USENIX Security (2026)
32. Zamani, M., Movahedi, M., Raykova, M.: RapidChain: Scaling blockchain via full sharding. In: CCS. pp. 931–948 (2018)